

第 31 讲练习题与答案

PyTorch Training Loop

练习题

1. 把下面五个动作按正确训练顺序排列：backward / step / forward / zero_grad / loss。
2. `loss.backward()` 执行后，Weight 一定已经改变了吗？真正改 Parameter 的典型语句是什么？
3. 为什么连续两次 backward 之间如果不清理梯度，`.grad` 可能变成两次贡献之和？
4. 如果 `parameter.grad is None`，直接执行最简单场景下的 `optimizer.step()`，Optimizer 是否拥有本轮梯度方向？
5. 一个循环每轮都执行 `zero_grad -> forward -> loss -> backward`，但从不 `step()`。预测和 Loss 会因为重复循环自动变好吗？为什么？
6. `model.eval()` 和 `torch.no_grad()` 有什么不同？
7. 模型训练 Loss 从 10 降到 0.001，能否直接推出 Test Set 泛化很好？
8. 在 LLM 预训练中，把普通回归例子中的 MSE 替换成什么常见 Loss？Training Loop 的基本闭环是否改变？
9. 为什么在研究训练失败问题时，检查 `parameter.grad is None` 有价值？
10. 请不用看答案写出一个最小 PyTorch Training Loop（5 行核心即可）。

答案

1. zero_grad -> forward -> loss -> backward -> step。
2. 不一定，而且标准情况下没有改变。backward() 主要计算 Gradient；optimizer.step() 才按更新规则修改 Parameter。
3. 因为 PyTorch 默认对 Parameter 的 .grad 做累加，而不是每次自动覆盖。这样也支持有意的 Gradient Accumulation。
4. 没有。本轮没有通过 backward 生成新的梯度信息；实验中 grad 为 None 时 step 没有修改 Weight/Bias。
5. 不会。每轮虽然重新算出了相同 Gradient，但 Parameter 没有更新，所以同样输入、同样 Parameter 会产生同样预测和 Loss。
6. model.eval() 切换 Module 的 evaluation mode，会影响 Dropout/BatchNorm 等层；torch.no_grad() 是关闭 Autograd 梯度记录。二者职责不同。
7. 不能。训练 Loss 小只说明在训练目标上拟合/优化得好；Generalization 需要 Validation/Test 等独立证据。
8. 常见是 Cross Entropy / next-token prediction loss。核心闭环仍然是 zero grad、forward、loss、backward、step。
9. 它可以帮助发现某个 Parameter 没参与当前 Loss、梯度路径被 detach/cut、参数没有正确注册或训练逻辑错误等问题。
10. 例如：

```
optimizer.zero_grad()
pred = model(x)
loss = loss_fn(pred, y)
loss.backward()
optimizer.step()
```